

Operating Systems

Homework Assignment #4

Due 24.12.2015, 23:55

In this assignment, you will use tools you've learned (system calls, signals, threads & processes) to create a simple shell program. For your shell, learn and use the following: **execvp** (man 3 execvp, *not to be confused with execv!*), **pipe**, **dup2** (man 2).

The program works by reading lines of input from the user:

- Each input line is treated as a command – a program name and its arguments
- Attached to this assignment is a code snippet which reads & parses input lines into an array “*arglist*” (*argv* for **execvp**).
- The *arglist* array is just a split of the input line into words (a word is a non-empty sequence of non-whitespace characters, where whitespace is space, tab, or newline). It is not necessarily the final argument for **execvp** (see below).
- Empty lines should be detected and ignored (also handled by the code snippet).

You are free to implement your shell as you wish, so long as it follows these guidelines:

- For each user command, execute the relevant process using **fork** and **execvp** (man 3 execvp, not *execv*).
- If a system call fails in the parent process, terminate the program with a proper error message. No need to notify anything to any running child processes. If a system call fails in a child process, output a proper error message and terminate the child process **only**. Nothing should change for the parent or other child processes.
- The user command might be empty (as explained earlier), or invalid! (e.g., a non-existing command/program). These can be treated as errors in the child process (i.e., it must *not terminate the shell*).
- Support background processes
 - If the last argument of the *arglist* array is “&” (ampersand only), run the child process in the background.
 - Do not pass this argument (&) to **execvp**!
 - The parent should not wait for the child process to finish, but instead continue executing commands.
 - To prevent zombies, use a new *thread* that will wait for the child process.
 - Following this, can we still use **wait** in our code? No, we must use **waitpid**. Make sure you understand why.
 - Assume background processes don't read input (*stdin*), and that “&” only appears as the last argument.
- Support nameless pipes
 - If *arglist* contains “|” (pipe only), run several child processes, correctly piping their *stdin* and *stdout* together.
 - You may create several child processes of the original shell, a hierarchy where each process is the parent of the previous one, etc. You're free to choose whatever option so long as it works correctly.
 - To pipe input and output of processes, use **pipe** and **dup2** system calls.
 - Use the same array for all **execvp** calls by referencing items in *arglist*, no need to duplicate, malloc, etc.
 - Assume no more than a single pipe (|) is provided, and it is correctly placed (at least 1 arg before and after).
- Assume pipes and background processes will not be combined, i.e., nameless pipes will never be run in the background (however, background processes might still be running from previous commands).
- **Bonus:** for implementing all of the following additions correctly (add a comment in your moodle submission):
 - Make sure the parent is not interrupted when a child process is running (only the child should terminate)
 - For non-background processes, make sure only child processes are interrupted by **SIGINT**.
 - Be sure to ignore **SIGINT** correctly (recall that **fork** duplicates a process, including its signal handlers).
 - When the child process finishes, restore the signal to the previous (not necessarily default!) handler.
 - Correctly support any number of pipes in *arglist*. This means you might need to create a chain of pipes.
 - Handle incorrectly-placed ampersand (&) and pipe (|) symbols (output a simple error message).
 - Correctly support the combination of piped and background processes (e.g., **ls -la | grep 'A' &**), including all combinations and possible errors.

Guidelines

- If you are unsure how something should work, base it on the existing terminal and shell (konsole & bash) in your VM. *If your shell behaves like the standard shell – it is correct!*
- If you're still unsure – ask in the forums.
- **As always**, closely follow the forums and all questions & answers provided there. Explanations, guidelines and relaxations may be given there, and they are **mandatory**.
- **Provided code file (*input.c*):**
 - The given code file contains your *main* function. You should create another file (***myshell.c***) implementing the *process_arglist* function.
 - **Do not** modify the given code file, and **do not** submit it. Only use it with your own code (in ***myshell.c***).
 - **Be careful** with the provided code. Recall that you need to free *arglist* and *line* in **all** child processes as well.
 - The original (shell/parent) process should **always** return 1 from *process_arglist*, and **all** child processes should return 0. This will make sure the original process continues processing user commands, while the child processes still free the allocated memory.
 - Recall that on error, you do not have to exit cleanly – you may terminate the child or parent processes (depends on where the error originated) without freeing memory.
- **Submit** a single C file: ***myshell.c***
 - **Document it** properly – with a main comment at the beginning of the file, and an explanation for ***every non-trivial*** part of your code. Help the grader understand your solution and the flow of your code.